Complete guide for integrating MPESA C2B to receive alerts for payments to your paybill/till

Updated April 12, 2025 • 11 min read

Shasa Thuo
I am a developer from Nairobi, interested in Python, Django, Javascript and React

I recently integrated M-Pesa's Customer-to-Business (C2B) API to enable a business to receive real-time alerts when customers make payments via Paybill. The primary goal was to automate service delivery upon payment. While Safaricom's Daraja documentation provides a foundation, I encountered gaps that required additional research. This guide documents my experience to assist others undertaking similar integrations.

Step one: DARAJA ACCOUNT

Step 1: Setting Up a Daraja Account

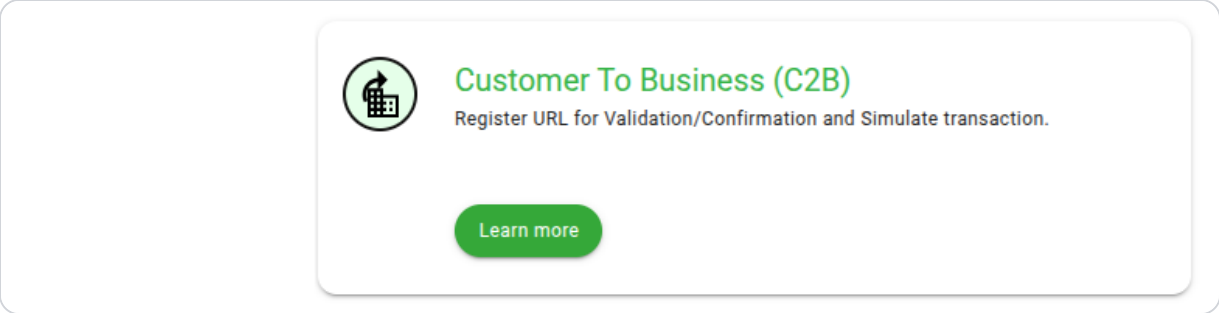
1. **Create an Account**: Visit Safaricom Developer Portal and register.
2. **Create an App**: After logging in, create a new app to obtain your `Consumer Key` and `Consumer Secret`.

Contents

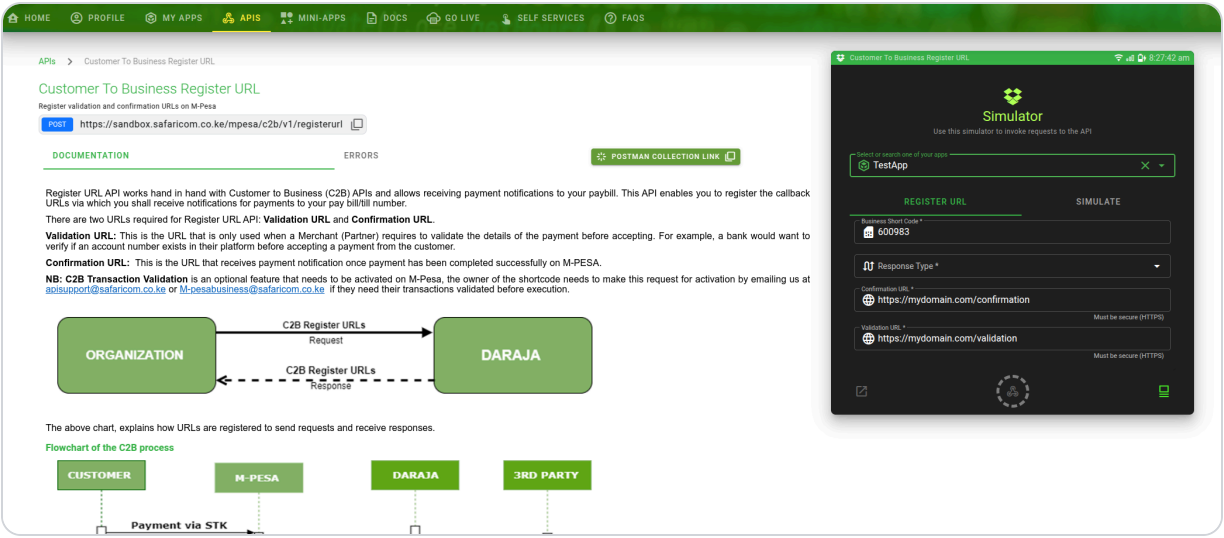
3. **Access C2B API**: Navigate to the APIs section and select "Customer to Business" to access **the C2B documentation.**

The C2B documentation allows you to simulate registering callback URLs and M-Pesa payments, enabling you to observe how Safaricom sends information after a customer has paid. This facilitates the development of backend logic to record transactions and assign goods/services to customers.Copy the consumer key and consumer secret as they will be needed in subsequent steps,

On daraja, click on apis, then select customer to business



It should take you to the documentation page for c2b



The c2b documentation page allows you to simulate registering a callback paybill url, after you have registered the paybill it also allows you to simulate mpesa payments, after which you can observe how safaricom

sends the infor after a customer has paid and can modify your backend logic to record the transaction and assign goods/services to that customer

Step two: Write the code

With sandbox test account credentials, proceed to create endpoints to register the callback URL and handle incoming notifications from Safaricom.

Obtaining an Authorization Token

To register the URL, you first need an authorization token from Safaricom. Send a request with your `Consumer Key` and `Consumer Secret` to the following URLs:

- **Sandbox**: `https://sandbox.safaricom.co.ke/oauth/v1/generate?grant_type=client_credentials`
- **Production**: `https://api.safaricom.co.ke/oauth/v1/generate?grant_type=client_credentials`

Use the sandbox URL for testing and the production URL when going live.

Registering the Paybill URL

After obtaining the authorization token, send a request to register your Paybill URL:

- **Sandbox**: `https://sandbox.safaricom.co.ke/mpesa/c2b/v1/registerurl`
- **Production**: `https://api.safaricom.co.ke/mpesa/c2b/v2/registerurl`

Note: The production URL uses `v2`, which differs from the `v1` version provided in Safaricom's production credentials email. Ensure you use the

correct version to avoid issues.

We then set up the code to handle the callback, which wil be called by safaricom when a customer pays.

The first code snippet takes consumer key and consumer secret and then sends a request to **https://sandbox.safaricom.co.ke/oauth/v1/generate?grant_type=client_credentials** to get the access token and attaches it to the request object for subsequent use.

```
const generateToken = async (req, res, next) => {

    console.log("KEY", {consumerKey, consumerSecret})

    const auth = Buffer.from(`${consumerKey}:${consumerSe
        "base64"
    );

    try {
        const authresponse = await fetch(auth_link, {
            method: "GET",
            headers: {
                Authorization: `Basic ${auth}`,
                "Content-Type": "application/json",
            },
        });

        if (!authresponse.ok) {
            throw new Error(
                `Auth request from safaricom failed with
            );
        }

        const data = await authresponse.json();
        const accessToken = data.access_token;
```

```
        req.token = accessToken;
        next();
    } catch (err) {
        console.error("Error generating auth token:", err
        const error = new Error(`Error generating auth to
        error.status = 400;
        return next(error);
    }
};
///
```

The snippet is a middelware function, it takes the consumer key and consumer secret and sends a request to **https://sandbox.safaricom.co.ke/oauth/v1/generate?grant_type=client_credentials** , it then puts that token in the request object that will be handled by the register paybill/till endpoint below.

```
const registerPaybillUrl = async (req, res, next) => {
    try {
        const {confirmation_url, validation_url} = req.bo
        if(!confirmation_url || !validation_url) throw ne
        const safRegisterUrl = safaricomCredentials.SAFAR
        const paybillNumber = Number(safaricomCredentials

        const requestBody = {
            ShortCode: paybillNumber,
            ResponseType: "Completed",
            ConfirmationURL:confirmation_url,
            ValidationURL: validation_url
        };
```

```
        const response = await fetch(safRegisterUrl, {
            method: "POST",
            headers:{
                "Content-Type": "application/json",
                "Authorization": `Bearer ${req.token}`
            },
            body: JSON.stringify(requestBody)
        });

        const data = await response.json();

        if (data.ResponseDescription === "Success") {
            return res.json({ status: "success", details:
        } else {
            console.log("Failed:", data);
            return res.status(400).json({ status: "error"
        }

    } catch (error) {
        next(error);
    }
};
```

This is a controller function that gets the token passed in my the middelware function earlier, it then calls safaricom's register paybill urls to get the paybill registered. the body of the request to the register paybill endpoint contains these values

- ShortCode: this is your paybill or till number
- ResponseType: this can be Completed or Cancelled, we only need to use Completed
- ConfirmationURL: this is the url of your callback endpoint, so you need to write the logic for how to handle the payment notifications

- ValidationURL: safaricom requires this in order to enable a feature where it can send a validation request that lets you validate if a customer's payment should be completed or cancelled. This may be nescesarry if you want to confirm a customer exists in your system before accepting their payment. We dont really need the validation url to complete the integration but safaricom requires it when sending the register paybill request so we have to add it.

Confirmation url controller logic

```
const paybillCallback = async(req, res, next)=>{
    try {

        res.json({
            ResultCode:0,
            ResultDesc:"Success"
        })
        const callbackData = req.body

        // process callbackData and save in your db or as
        // the account details will be in callbackData.Bi


        console.log({
            payment:callbackData
        })

        return

    } catch (error) {
        next(error)
    }
}
```

When the controller receives a request, it sends back to safaricom a successful response with the structure outlined in the daraja docs and then processes the callback data.

The callback data that you receive from safaricom looks like this,

```
{
   "TransactionType": "Pay Bill",
   "TransID":"RKTQDM7W6S",
   "TransTime":"20191122063845",
   "TransAmount":"10"
   "BusinessShortCode": "600638",
   "BillRefNumber":"invoice008",
   "InvoiceNumber":"",
   "OrgAccountBalance":""
   "ThirdPartyTransID": "",
   "MSISDN":"25470****149",
   "FirstName":"John",
   "MiddleName":""
   "LastName":"Doe"
}
```

BillRefNumber is the details the customer put in the account when paing via paybill, If you customers ented their account id, you can know which customer has paid

MSISDN is the phone number, however it will be hashed. This is to protect the customer's privacy. and avoid cases where companies spam customers with messages after they have paid with the paybill. To get the phone number you must have either existing customer numbers in your database, you can hash them and compare the hash with the value in MSISDN to know which customer has paid or use a service like **https://decodehash.com/app** where you send them the MSISDN and they respond with the phone number

We also need validation url so lets write the code for that endpoint

```
const paybillValidation = async(req,res,next)=>{
    try {
        return res.json({
            "ResultCode": "0",
            "ResultDesc": "Accepted",
        })
    } catch (error) {
        next(error)
    }
}
```

All it does is send back the resultcode and description, the url for this controller won't be called unless you have activated validation by requesting that your paybill callback url have validation from safaricom. Its somewhat redundant, however since we need to have the validation url when registering the callback url we write the logic for it and if the need arises for validation we can expand the logic in this controller.

Wit the middleware and controllers in place we can create the routes

```
const express = require("express");
const router = express.Router();
const {
    paybillCallback,
    registerPaybillUrl,
    paybillValidation
} = require("../controllers/darajaApis")
const {generateToken} = require("../middleware")


router.post(`/paybillcallback`, paybillCallback);
```

```
router.post(`/validate`,paybillValidation);
router.post(`/registerpaybill`, generateToken, registerPa


module.exports = router;
```

- /paybillcallback will be called by safaricom when a customer makes a payment
- /validate is where you can validate a customer and if their payment should proceed or be cancelled if you have that option enabled
- /registerpaybill this is the url created so we can send the register paybill request to safaricom so that they can begin sending data to /paybillcallback. Notice how when a request comes in to /registerpaybill we first get the auth token from the middleware we created earlier and then pass on the request to registerPaybillUrl which sends the actual paybill url registration request

After the routes we import them into index.js which is the endtoy point for the nodejs server

```
const express = require("express");
const mpesaRoutes = require("./routes/mpesaRoutes")

const port = process.env.PORT || 3000;

app.use(express.json());

const apiVersion = '/api/v1/';

app.use(`${apiVersion}/`,  mpesaRoutes)
```

```
app.listen(port, () => {
    console.log(`The server is up and listening on port $
});
```

With this setup you app urls should be

- www.example.com/api/v1/paybillcallback
- www.example.com/api/v1/validate
- www.example.com/api/v1/registerpaybill

Now we are ready for the next step

Step three: Local Testing Setup/Ngrok and postman

We need postman to test our apis and ngrok so that we can create a https secure tunnel to our localhost app that we can use to test on the daraja website. So go ahead and install ngrok and postman if you do not already have them. I will not go over their installation here, their installation is relatively straightforward.

Once you have them installed start the nodejs server, which should be running on localhost:3001 and then run Ngrok and point it to the 3001 port. If you are on linux the command is

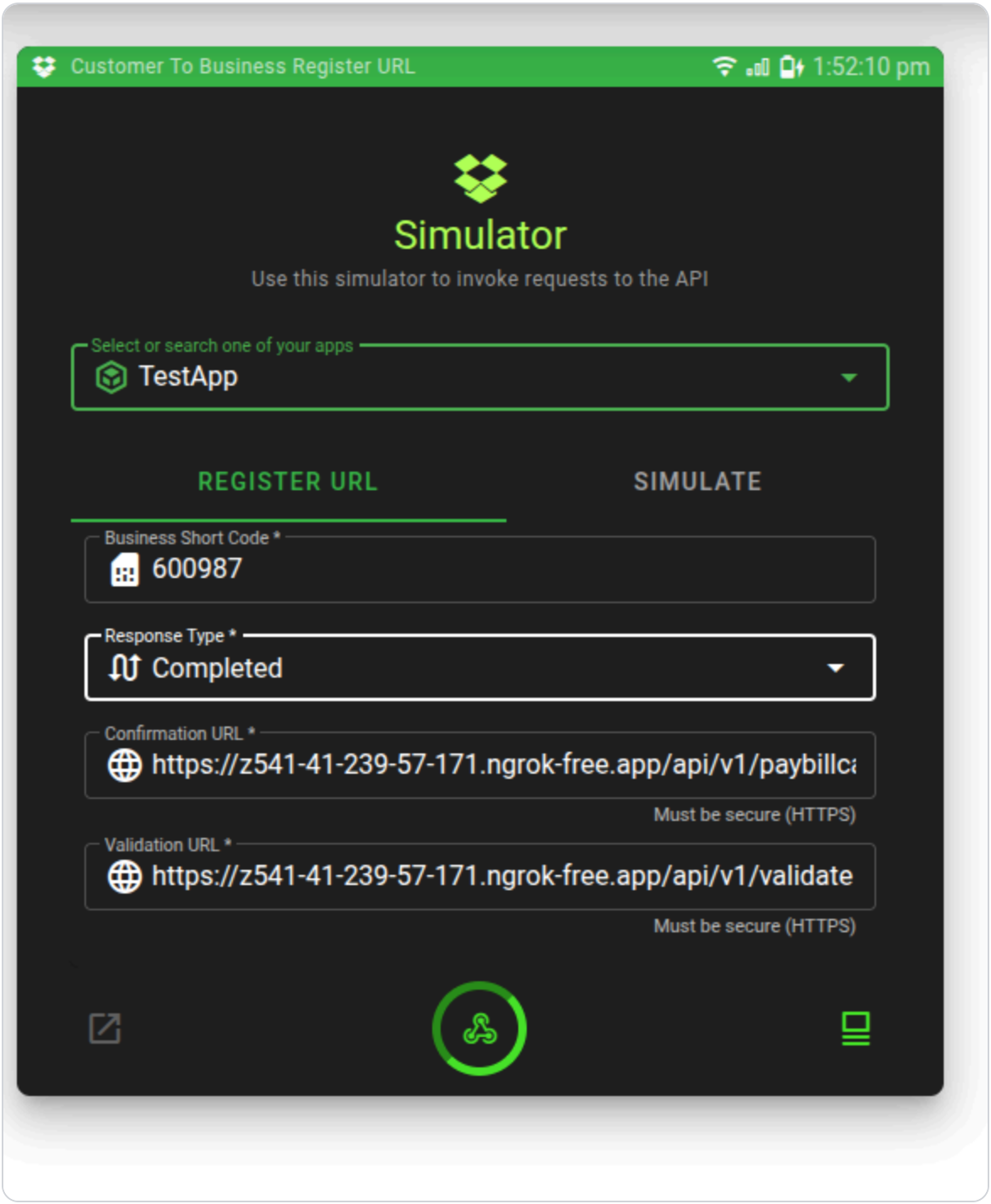
```
$ngrok http http://localhost:3001
```

After starting the app and activating ngrok, the endpoint to test the apis will be something like **https://z541-41-239-57-171.ngrok-free.app**. We can then

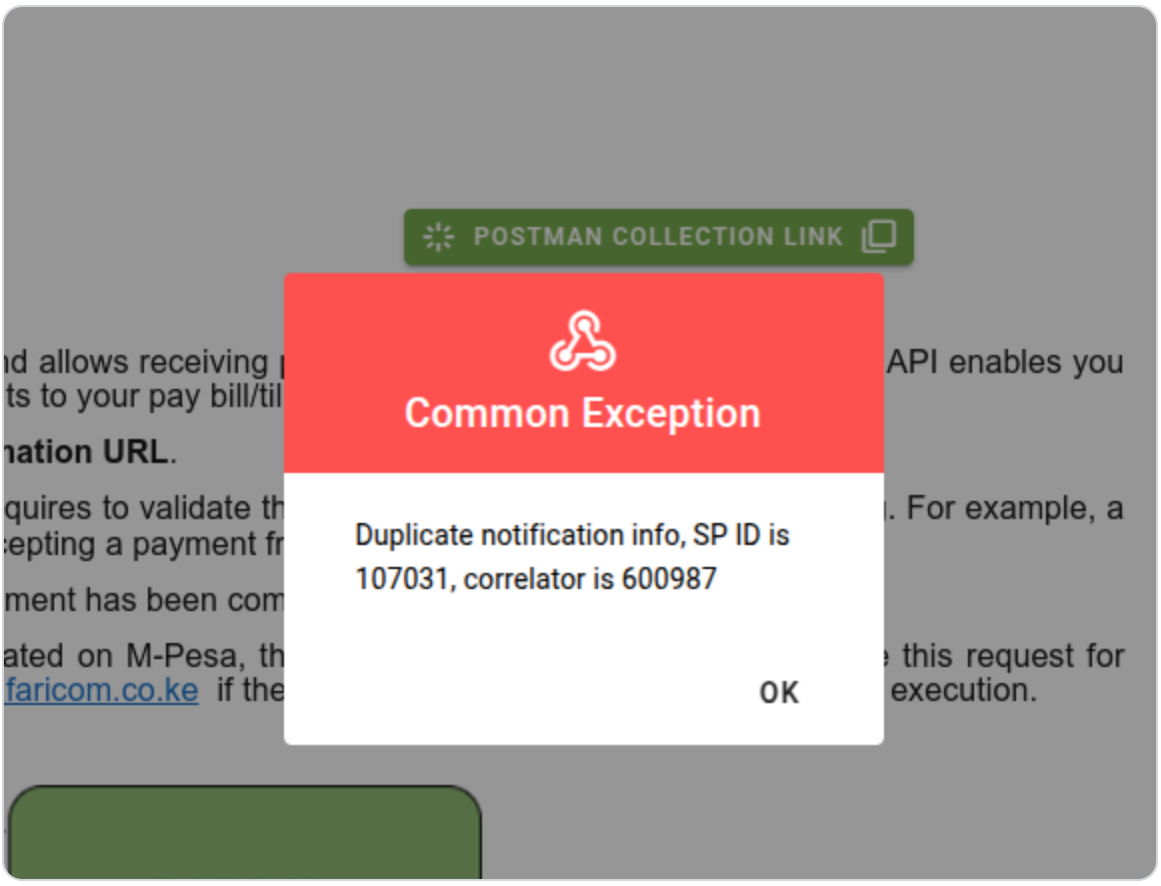
use this api to test out the c2b integration logic on daraja. When going live you need to use your actual https endpoint that you get when you register your domain. Safaricom does not allow ngrok on live endpoints.

Also important to note is that Daraja is very flaky when sending callbacks for the sandbox urls. Receiving callbacks might never work on sandbox or only work 40 percent of the time. The consensus amongst developers is that you need to make your app live in order to test the c2b integration because the live services works more reliably than sandbox. In my case I was able to get callbacks using daraja so I include that process in this article.

Now that you have the ngrok url pointing to your localhost nodejs instance go to this section on daraja under c2b apis and enter the url you got from ngrok for he callback url and valdate url.

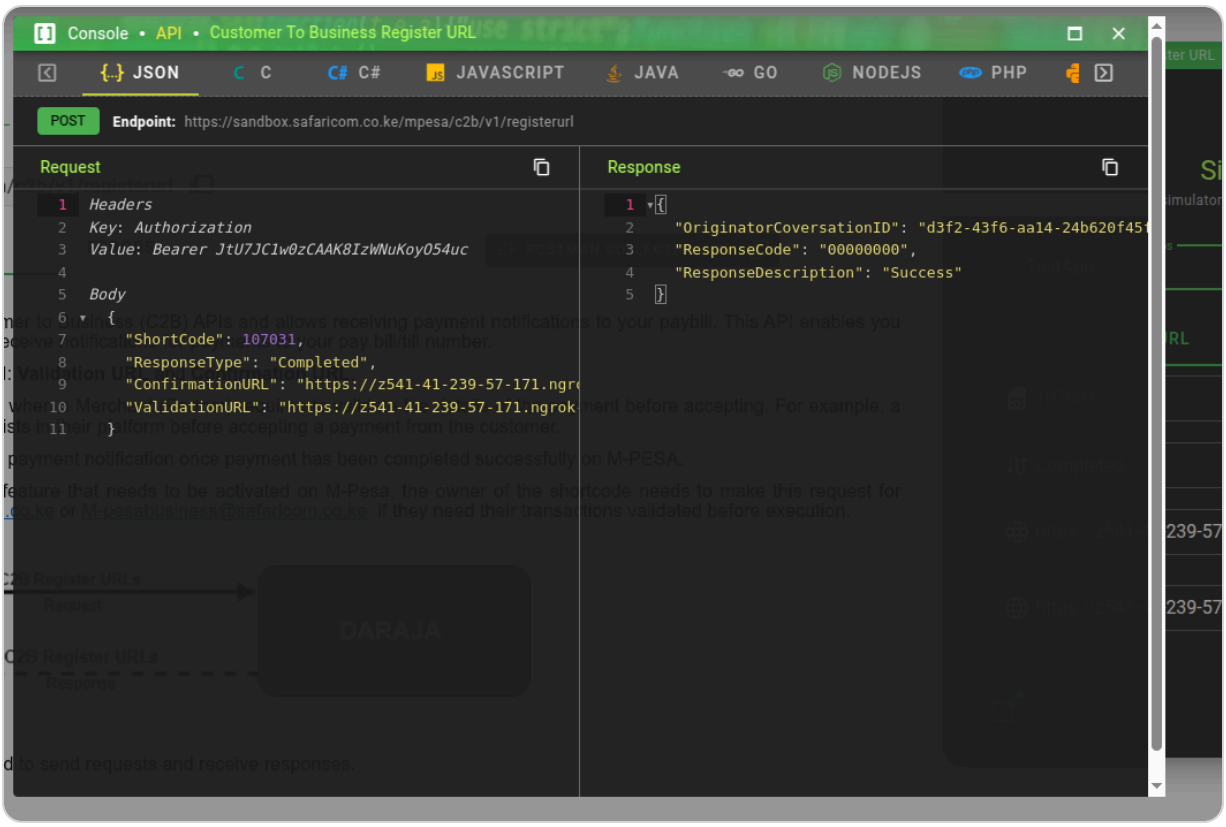


When you click submit you will probably get this error



In which case replace the business shortcode with the sp id 107031, or whatever the error prints out and you should be fine.

On successful submission of urls on daraja this will pop up



Below is the response that you get when successfully submit c2b urls on your live production urls. Getting this means that the urls you submitted are now registered and you should begin receiving callbacks on your /paybillcallback endpoint whenever someone makes payments to your paybill.

```
{
  "OriginatorCoversationID": "d3f2-43f6-aa14-24b620f45f3c
  "ResponseCode": "00000000",
  "ResponseDescription": "Success"
}
```

And this is how you should structure the register paybill request, we already covered this above when creating the /registerpaybill endpoint

```
{
  "ShortCode": 107031,
  "ResponseType": "Completed",
  "ConfirmationURL": "https://z541-41-239-57-171.ngrok-
  "ValidationURL": "https://z541-41-239-57-171.ngrok-fr
}
```


After that click the simulate tab

When you click the green circular submit button, you should receive a callback on your ngrok urls that point to your localhost apis. However its not guaranteed that the callback will happen, you may never receive a callback at all. The only way to test reliably is to deploy the nodejs server and get live https endpoints after getting a certificate for your domain.

This is the structure of the data you get back from safaricom on the /paybillcallback endpoint when you simulate a payment via daraja or when you have the live endpoint deployed and receive callbacks from actual payments through your paybill. The structure and how to handle the response is discussed in step two.

```
{
    "TransactionType": "Pay Bill",
    "TransID":"RKTQDM7W6S",
    "TransTime":"20191122063845",
    "TransAmount":"10"
    "BusinessShortCode": "600638",
    "BillRefNumber":"invoice008",
    "InvoiceNumber":"",
    "OrgAccountBalance":""
    "ThirdPartyTransID": "",
    "MSISDN":"25470****149",
    "FirstName":"John",
    "MiddleName":""
    "LastName":"Doe"
}
```

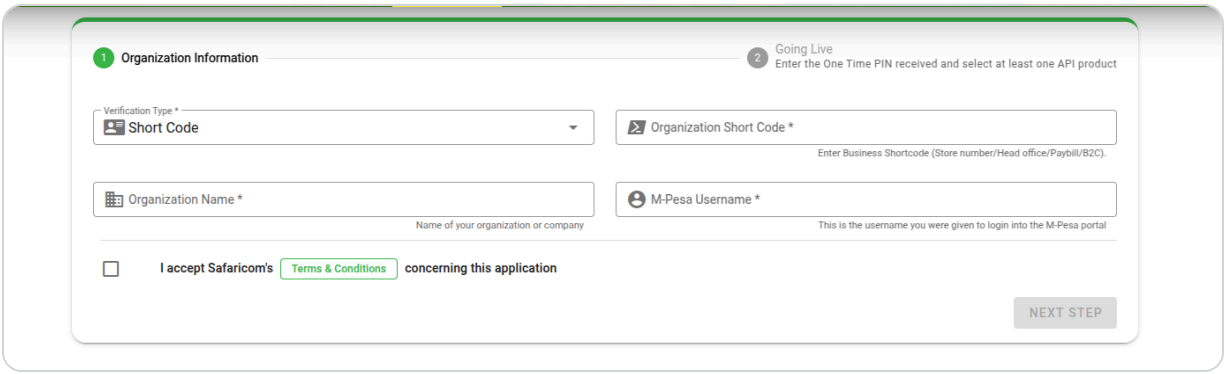
Step four: mpesa admin account creation and going live on daraja

At this point you have refined your logic and made sure that you can correctly receive payment notifications, so you are ready to go live, or you have not had much success getting daraja to call your apis and need to go live anyway in order to test your actual production callback urls.

For the paybill or till number you intend to integrate with c2b you should have access to **https://org.ke.m-pesa.com/** where you have admin access over the payment shortcode. If you don't have access, email safaricom helpdesk and request its creation/activation.

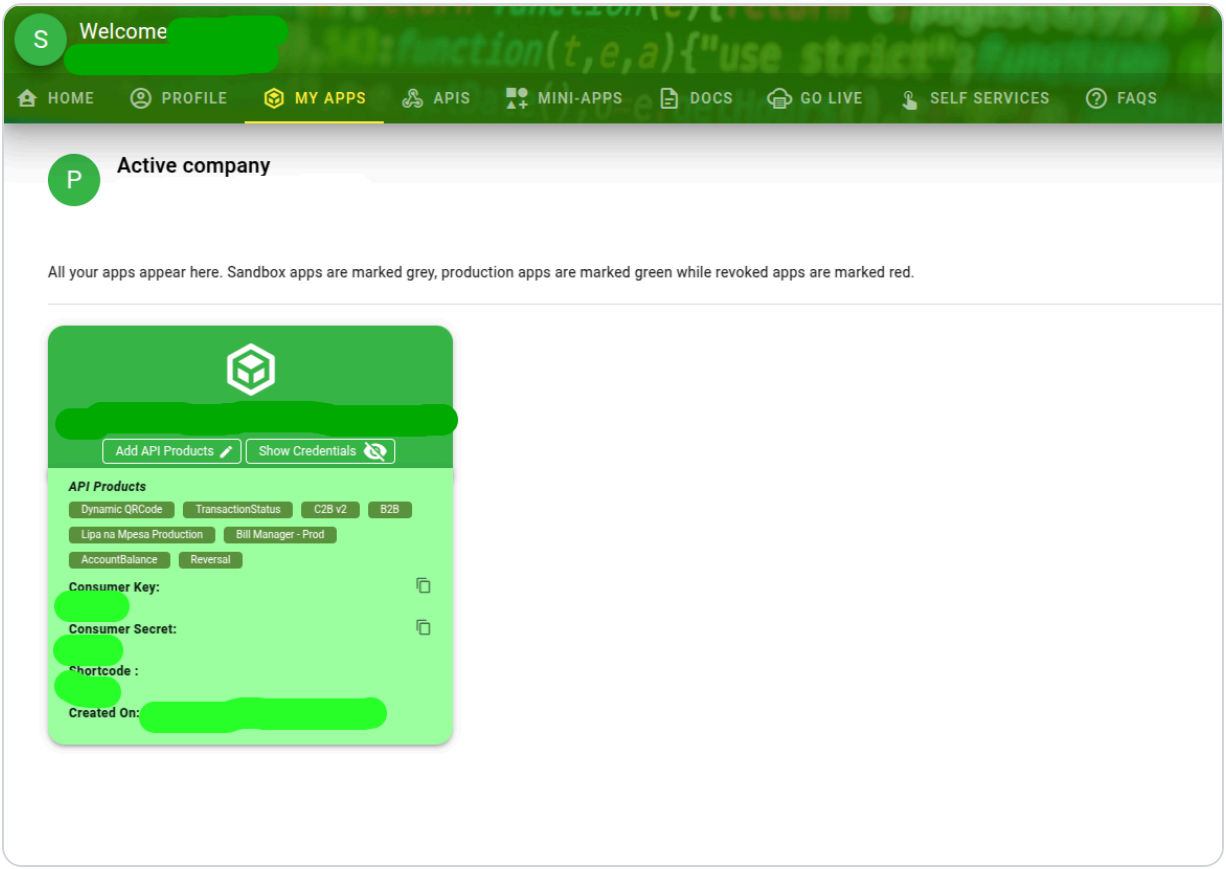
In **https://org.ke.m-pesa.com/** create a new user and assign the new user api creation permission role for your paybill/till. You may have to reset the new user's password in order for it to be active. If you encounter any hitches when creating an api user, don't hesitate to call the mpesa for business contacts **M-PESABusiness@Safaricom.co.ke** or 0722002222 or 2222. They will tell you what additional steps you need to take.

After getting the api user credentials for your paybill/till you can now go back to daraja and click the go live tab



You will get this form where you enter the credentials of the api user you created in **https://org.ke.m-pesa.com/** . After which you will get an otp on the contacts submitted for the api user on the org website. The shortcode is your paybill/till. Safaricom will create create a live app on daraja which looks like this.

The live app will appear green unlike the grey sandbox apps



You will also get two emails from Safaricom upon going live, one email will contain the passkey, which you dont need for c2b but will need if you also want to enable stk push api for your paybill/till(upcoming article), the other email from safaricom will have the production urls that you replace with the sandbox urls. However the c2b url in the email you get from safaricom **https://api.safaricom.co.ke/mpesa/c2b/v1/registerurl** wont work unless you replace v1 with v2 🤦🏾‍♀️.

Replace the consumer key and consumer secret with the ones for the live app and deploy your server. If you are self hosting you may use something like certbot and letencrypt to get a certificate for your domain that enables https or or if you are using a provider like vercel the https activation process for your domain is automated.

When your https endpoint is ready, you can go ahead and call the /live registerpaybill endpoint using postman to register your live callback url with safaricom. After which you should begin receiving payment notifications.

#mpesa #daraja-api #nodejs #payments

Comments Join the discussion

No comments yet. Be the first to comment.

More from this blog

Debugging Disk Space Issues on Linux

Some handy commands for quickly checking whats eating up storage space on linux vps(in my case it was poorly configured github action runners and chonky npm and nextjs cache) Get a High-Level Overview with df The df command (Disk Free) gives you a su...

Apr 22, 2025 · 2 min read

🚀 How to Increase Swap Space on Ubuntu (DigitalOcean Droplet Friendly)

If you're running a server with limited RAM — like a 2GB DigitalOcean droplet — you might run into memory issues when building large applications (hello oom-kill). A quick and effective solution is to increase your swap space. This guide walks you th...

Apr 16, 2025 · 2 min read

How to use let's encrypt and Certbot to point a domain name to your server IP address

So you are at the point in your project where you have successfully deployed your backend app to a server like EC2 and have nginx as a reverse proxy or you have configured your nginx to serve the static frontend files so that going to an ip like http...

Dec 21, 2024 · 2 min read

How to create a linux systemd service and timer that activates a script every few seconds

You have a bash script on your linux installation that you would like to run periodically, it could be anything, for instance you have a script that takes periodic snapshots of the database and performs a backup, or a script that pulls in changes fro...

Jun 16, 2024 · 4 min read

Learning journey

18 posts

Sharing knowledge and learning about Python, Javascript, Django and React